Drift-Aware Bike-Share Demand Forecasting

A versioned and monitored ML pipeline on Citi Bike (New York City) trip data

Prepared by	Om	**Document**	Project proposal, v1.0
Course	MLOps (final-year project)	**Date**	23 October 2026
Planned duration	8 weeks (adjustable)	**Infrastructure cost**	USD 0 (open data, open-source tools)

Summary

Bike-share operators must move bikes between stations ahead of demand: an empty station loses a rental and a full station blocks a return. Rebalancing depends on short-term demand forecasts, and a forecasting model trained once degrades as seasons, weather patterns and rider behaviour change. The degradation stays invisible unless predictions are compared with ground truth that arrives later. This project builds a 24-hour station-level demand forecasting service for Citi Bike and, as its core contribution, the controls that keep it trustworthy: **data versioning** (every dataset is reproducible), **model versioning** (every model is traceable to the exact data and code that produced it, with controlled promotion and rollback) and **monitoring** (drift and live accuracy tracked against delayed actuals). Everything uses public data and free tools. Section 2 explains the business significance and, just as importantly, where that claim stops.

1. Problem statement

A one-time forecasting notebook cannot answer the questions an operator actually has: *Is the model still accurate this month? Which data and code produced the model now in service? Can we revert safely if a new model is worse?* The problem addressed here is the full lifecycle of a demand model whose inputs drift with season and weather, not a single accuracy score.

2. Industry and business significance

Short answer: yes, the problem is a real operational one, but the value is conditional and this project does not claim a dollar figure. Station-based bike-share systems must keep bikes and free docks where riders need them. Operators do this by rebalancing, moving bikes between stations, and the research literature describes rebalancing as a cost driver that grows as systems expand. Most systems rebalance in a planned way overnight, when demand is low, and those plans depend on target inventory levels that are in turn derived from demand forecasts. A better and more trustworthy forecast therefore feeds directly into service availability and operating effort.

2.1 Who benefits

Stakeholder	Interest in the outcome
Bike-share operators	Fewer empty or full stations per unit of rebalancing effort; a defensible basis for overnight rebalancing plans
City transport authorities	Operators report service data to city agencies (Citi Bike publishes monthly operating reports for the NYC Department of Transportation), so forecast quality and auditability support accountability
Riders	Bikes available at the start of a trip and docks available at the end
Data and platform teams	Reproducible, monitored models that can be audited, rolled back and handed over

2.2 How forecast errors turn into business cost

Forecast error	Operational consequence	Project response
Under-forecast demand	Station runs empty or full, rides are lost, riders are dissatisfied	Report **under-forecast shortfall** (total demand units under-predicted) next to MAE
Over-forecast demand	Bikes are moved to stations that did not need them, wasting rebalancing effort	Report MAE and RMSE; compare against the seasonal-naive baseline
Silent decay over time	Plans keep being made from a model that no longer fits current demand	Rolling-error monitoring against delayed actuals and drift alerts

Because the cost of under- and over-forecasting is rarely symmetric, MAE alone can hide what matters to an operator. The shortfall figure is a *proxy* for service risk, not a monetary estimate.

2.3 Why the MLOps controls matter to this business

Business risk	Control in this project
Forecast quality drifts as seasons and weather change, unnoticed	Monitoring: input drift and rolling error versus actual demand (Section 6.5)
Nobody can say which data and code produced the model behind last night's plan	Model versioning with Git commit and dataset hash on every registered model (Section 6.4)
A bad model update disrupts overnight operations	Promotion gate and one-step rollback through registry aliases
Results cannot be reproduced for a manager, auditor or city agency	Dataset versioning with DVC and a reproducible pipeline (Section 6.3)

The general point that the model is only a small part of a production ML system, and that data dependencies, monitoring and configuration carry ongoing maintenance cost, is argued in Sculley et al. (2015).

2.4 Limits of the significance claim

- **Public data shows trips, not availability.** Trip files record rides that happened. Where a station was empty, demand went unrecorded, so the target understates true demand (censored demand). The project cannot compute real cost savings or lost-ride revenue.
- **The value of forecasting is studied, not assumed.** Researchers explicitly ask whether demand forecasting is worth its cost for rebalancing (Ataç et al.), so the benefit depends on how the operator rebalances.
- **What the project does claim:** a reproducible, monitored forecasting pipeline of the kind such operations need, evaluated on forecast error and a shortfall proxy. A natural follow-on, out of scope here, is feeding the forecasts into a rebalancing simulation to estimate operational impact.
- **Transferability (author's assessment, not evidence-backed here):** the same pattern of versioned data, a model registry and drift monitoring with delayed labels applies to other demand-forecasting settings.

3. Objectives

1. **Forecast** hourly departures for the busiest stations over the next 24 hours, beating a seasonal-naive baseline.
2. **Version data:** every dataset version is reproducible from a Git commit, and any model can be traced to it.
3. **Version models:** every run records parameters, metrics, code commit and data hash; promotion and rollback are controlled through registry aliases.
4. **Monitor:** track input drift and rolling forecast error against actual demand that arrives after the forecast.
5. **Demonstrate a closed loop** (detect, retrain, compare, promote, roll back) on real months of data at zero cost.

4. Scope

In scope	Out of scope
Hourly departure forecasts for the 30 busiest stations (fixed list chosen from dataset v1) DVC data versioning, MLflow tracking and registry, Evidently monitoring FastAPI service in Docker Compose (local deployment) GitHub Actions CI and a retrain workflow Replay of real months to simulate production time	Kubernetes or cloud deployment Feature store Live streaming from the GBFS real-time feed Truck routing or rebalancing optimisation Deep learning models

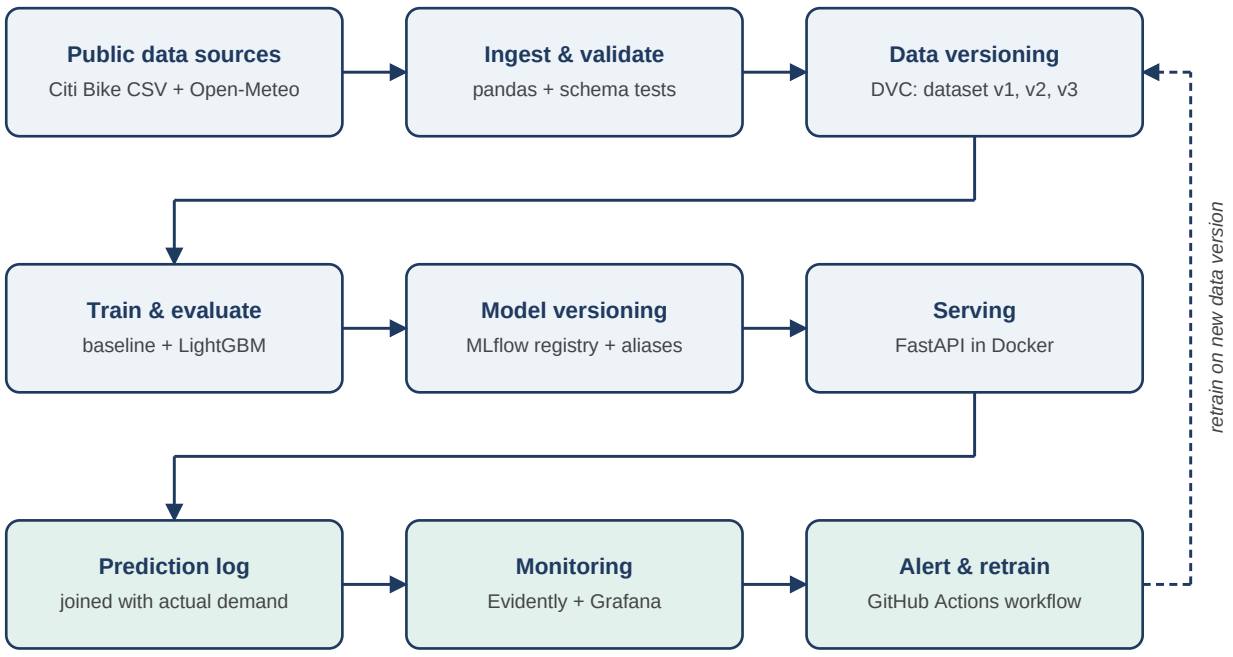
5. Data sources (public, zero cost)

Source	Content	Access	Cost and terms
Citi Bike trip histories	Trip start and end time, stations, rider type, bike type	Downloadable files from citibikenyc.com/system-data; published monthly in recent years	Free; governed by the NYCBS Data Use Policy
Open-Meteo Historical Weather API	Hourly temperature, precipitation, wind	REST endpoint (archive-api.open-meteo.com), no API key	Free for non-commercial use; CC BY 4.0 attribution; fair-use limits (reported around 10,000 calls/day)
US holiday calendar	Public holiday flags	Python *holidays* package	Open source, no cost

Open-Meteo responses are fetched once per month-chunk and cached as DVC-tracked files, so the API is called a handful of times in total and results stay reproducible.

6. Proposed solution

6.1 Architecture



Layer	Tool	Role	Cost
Data versioning	DVC + Git	Versioned datasets and reproducible pipeline stages (*dvc.yaml*)	Free, open source

Layer	Tool	Role	Cost
Experiment and model registry	MLflow (local server, SQLite backend)	Run tracking, model registry, champion/challenger aliases	Free, open source
Model	scikit-learn, LightGBM	Baseline, Ridge and gradient-boosted candidates	Free, open source
Serving	FastAPI, Docker Compose	Loads the champion model; logs every prediction	Free, open source
Monitoring	Evidently; Prometheus + Grafana (stretch)	Drift and performance reports; service dashboard	Free, open source
Automation	GitHub Actions	CI checks and retrain workflow	Free for public repositories

6.2 Forecasting task

- **Target:** number of departures per station per hour. **Horizon:** next 24 hours, issued once per day at 00:00. Ground truth for each forecast is therefore known 24 hours later.
- **Features:** hour, weekday, month, holiday flag, temperature, precipitation, wind, demand lagged 24 h and 168 h, and a 7-day same-hour mean. Every feature is computed only from information available at issue time.
- **Models:** seasonal-naive baseline (demand one week earlier), Ridge regression, and LightGBM with several hyperparameter sets.
- **Validation:** time-ordered splits only, no shuffling. Primary metric MAE; secondary RMSE and under-forecast shortfall.

6.3 Data versioning (DVC)

Raw monthly files, cached weather responses and processed feature tables are DVC artifacts. The pipeline (ingest, features, train) is declared in *dvc.yaml*; each dataset version is a Git commit whose *dvc.lock* pins content hashes. Fourteen consecutive months (M1 to M14, chosen in week 1 to span a seasonal transition) are released in three steps to mimic data arriving over time:

Version	Training data	Replay window (treated as production)
v1	M1 to M8	M9 to M10
v2	M1 to M10	M11 to M12
v3	M1 to M12	M13 to M14

6.4 Model versioning (MLflow)

- Every run logs parameters, metrics, feature list, Git commit, dataset version (DVC hash) and the model artifact.
- The best run is registered as **challenger**. The service always loads the **champion** alias, so promotion and rollback are alias changes, not code redeployments.
- **Promotion gate:** a challenger replaces the champion only if its MAE on the latest holdout month is lower by a configured margin (initially 3%). The margin is a config value to be tuned, not a claim about expected results.

6.5 Monitoring

During replay the service issues a forecast each day; actual demand is joined to the prediction log 24 hours later. The reference dataset is the training data of the current champion.

Layer	Signal	Method	Action
Input drift	Weather, calendar and lag features versus reference	Evidently data drift report (per-feature tests, share of drifted features)	Flag when drifted share exceeds a configured threshold
Prediction drift	Distribution of predicted demand	Evidently	Informational
Model performance	Rolling 7-day MAE versus reference MAE	Evidently regression report or custom metric	Alert when above 1.25x reference (configurable)
Service health	Latency, request count, errors	Prometheus + Grafana (stretch goal)	Informational

6.6 CI/CD and automation

- **On every pull request (GitHub Actions):** lint, unit tests, data schema checks and a training smoke test on a small sample fixture.
- **Retrain workflow:** started manually or by a monitoring alert; it runs the DVC pipeline on the newest dataset version, registers a challenger and reports the gate decision. A human approves the alias change.
- **Design decision:** full-size retraining runs locally with the same DVC pipeline because the DVC remote is a local folder; CI never needs paid storage.

7. Demonstration plan

1. Show the problem: the champion trained on v1 serves forecasts; display its lineage (Git commit and data hash).
2. Replay the next window: monitoring shows input drift and a rising rolling MAE.
3. Release dataset v2 and run the retrain workflow; compare challenger and champion in MLflow.
4. Apply the promotion gate and switch the champion alias; the service now serves the new model.
5. Trigger a rollback by repointing the alias and show service recovery without a code change.

8. Implementation plan (8 weeks)

Week	Work	Outcome
1	Problem framing, data audit, repo scaffold, confirm file availability for M1 to M14, DVC init	Repo, data plan, evaluation protocol
2	Ingestion, weather caching, feature pipeline, schema tests	Dataset v1 tracked in DVC
3	Baseline and LightGBM training, MLflow tracking	First registered champion
4	FastAPI service, Docker Compose, prediction log and **actuals join**	Serving with logged predictions
5	Evidently reports, thresholds, optional Grafana dashboard	Working monitoring on replayed data
6	Datasets v2 and v3, retrain workflow, promotion gate, rollback, GitHub Actions CI	Closed loop end to end
7	Lineage report, documentation, full dry run, fix-ups	Reproducible repository
8	Slides, demo rehearsal, buffer	Presentation and live demo

8.1 Project Gantt chart (week by week)

Project phase	W1	W2	W3	W4	W5	W6	W7	W8
Problem framing, data audit, repo and DVC setup								
Ingestion, weather caching, features, dataset v1								
Baseline and LightGBM training, MLflow, first champion								
Serving integration, prediction log, actuals join								
Monitoring: Evidently reports, thresholds, alerts								
Datasets v2/v3, retrain, promotion gate, rollback, CI								
Lineage report, documentation, end-to-end dry run								
Slides, demo rehearsal, buffer								

Each phase occupies its own week and the phases run one after another, with no overlap. The phase bar marks the week in which the phase is completed and demonstrated; what each phase delivers is listed in the table above.

8.2 Team roles and work distribution (three members)

Work is split by pipeline stage rather than by tool, so that each member owns a working part of the end-to-end loop and the lineage chain (data to model to service) stays connected. Member names are to be filled in by the team.

Role	Responsibilities	Owned deliverables
Member A Data and automation lead	Ingestion, weather caching, feature pipeline, DVC stages, dataset versions v1 to v3, replay driver, schema tests, GitHub Actions CI and retrain workflow	Versioned datasets, *dvc.yaml*, CI workflows, lineage and reproducibility report
Member B Modeling and registry lead	Baseline, Ridge and LightGBM, evaluation protocol (MAE, RMSE, shortfall), MLflow tracking and registry, champion/challenger aliases, promotion gate, rollback	Registered model versions, evaluation tables, promotion and rollback logic
Member C Serving and monitoring lead	FastAPI service, Docker Compose, prediction log, join of predictions to actuals, Evidently reports, drift and error thresholds, optional Grafana	Running service, monitoring reports and alerts, monitoring dashboard

8.3 Shared responsibilities and working rules

- **Shared by all three:** repository setup in week 1, integration in week 7, the README, the demonstration and the final presentation. Each member presents the pillar they own.
- **Interface contracts agreed in week 1:** feature table schema (Member A), registered model name and alias convention (Member B), prediction log schema (Member C). These let the three work in parallel.
- **Avoiding blocking:** Member B trains on a data sample until dataset v1 is ready in week 2; Member C serves a dummy or baseline model until a champion exists in week 3.
- **Cross-review:** every pull request is reviewed by a different member, so all three can explain every component in questions.
- **Weekly integration check:** a 15-minute run of the pipeline end to end from a clean clone.
- **Load balancing:** Member C carries the heaviest technical load; CI therefore sits with Member A, and if Member C falls behind, Member B takes over the Evidently reports.

8.4 Weekly plan by member

Week	Member A	Member B	Member C
1	Repo scaffold, DVC init, check file availability for M1 to M14, feature schema contract	Evaluation protocol, metric definitions, MLflow setup, model and alias contract	Docker Compose skeleton, prediction log schema, service stub with dummy model
2	Ingestion, weather caching, feature pipeline, dataset v1	Baseline and Ridge on a data sample	FastAPI endpoint and prediction logging with dummy model
3	Schema tests, dataset v1 finalised, CI lint and tests	LightGBM runs, first champion registered	Service loads the champion alias
4	Replay driver that feeds daily windows	Per-window evaluation, shortfall metric	Actuals join, rolling error computation
5	Dataset v2 release	Challenger runs, promotion gate logic	Evidently reports, thresholds, alerts
6	Dataset v3, retrain workflow in GitHub Actions	Rollback via aliases, gate tests	Monitoring over v2 and v3 replay, optional Grafana
7	Lineage report, fresh-clone reproducibility test	Registry export, results tables	Monitoring report export; all: end-to-end dry run
8	Slides for data versioning and lineage	Slides for models and results	Slides for monitoring; all: demo rehearsal

9. Success criteria

Criterion	Target	How it is checked
Forecast quality	Champion beats the seasonal-naive baseline on MAE on each replay window	MLflow metrics on held-out months
Business proxy	Under-forecast shortfall reported next to MAE for every replay window and every model version	Evaluation script, MLflow metrics
Lineage	100% of registered model versions carry Git commit and dataset version	Registry audit script
Reproducibility	A clean clone running *dvc repro* reproduces the registered model metrics within a small tolerance	Fresh-clone test
Detection	An alert fires within one monitoring window (7 days) of a degradation visible in the actuals	Replay logs and Evidently reports
Rollback	Reverting the champion alias restores service in under 5 minutes, no redeploy	Timed demo step

10. Risks and mitigations

Risk	Mitigation
Weather features are reanalysis values, which a live system would replace with forecasts (optimistic bias)	State it as a documented limitation; optionally test with forecast-based weather at serving time
Chosen months show little real drift	Select a window spanning a seasonal transition; report observed drift honestly; if needed, add a clearly labelled injected shift as a secondary test
Trip data understates demand where stations were empty (censored demand), so cost savings cannot be computed	State it explicitly (Section 2.4); claim forecast and pipeline quality only, no monetary savings
Monitoring join (predictions to actuals) is complex	Build it in week 4, before dashboards
Citi Bike file schemas or availability differ across months	Use only the monthly-published era; schema validation tests; confirm files in week 1
Open-Meteo terms or limits change	Cache responses in DVC; academic, non-commercial use; attribute CC BY 4.0
Members block each other or the load is uneven	Interface contracts in week 1, sample data and dummy model as stand-ins, cross-review, weekly integration check, fallback reassignment (Section 8.3)
Scope creep (Kubernetes, feature store)	Listed as out of scope; revisit only after the closed loop works

11. Deliverables

- Public Git repository with *dvc.yaml*, tests, Dockerfiles and GitHub Actions workflows.
- MLflow experiment and registry export showing lineage for every model version.
- Evidently reports (HTML) and, if time permits, a Grafana dashboard.
- Lineage and reproducibility report, plus a README that lets a reviewer run the demo.
- Contribution log in the README listing the responsibilities and pull requests of each member.
- Final presentation with live demo, structured as problem, evidence of decay, pipeline fix, before-and-after results.

References

Sculley, D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems 28*.

Citi Bike. System Data (trip histories and NYCBS Data Use Policy). https://citibikenyc.com/system-data

Open-Meteo. Free Weather API, Historical Weather API (CC BY 4.0). https://open-meteo.com

Ataç, S., Obrenovi■, N., Bierlaire, M. Vehicle sharing systems: Does demand forecasting yield a better service? Swiss Transport Research Conference (STRC), 2020. https://www.strc.ch/2020/Atac_EtAl.pdf

Predictive and Prescriptive Performance of Bike-Sharing Demand Forecasts for Inventory Management. arXiv:2108.00858. https://arxiv.org/pdf/2108.00858

Distributed forecasting and ant colony optimization for the bike-sharing rebalancing problem with unserved demands. *PLoS ONE* 14(12), 2019.

Modelling the Impacts of Rebalancing Strategies on Bike Share Toronto. Transportation Association of Canada. https://tac-atc.ca/wp-content/uploads/modelling_the_impacts_of_rebalancing_strategies_on_bike_share_toronto.pdf

Hersbach, H. et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999-2049.

DVC (dvc.org), MLflow (mlflow.org), Evidently AI (evidentlyai.com): official documentation, to be checked against current versions in week 1.